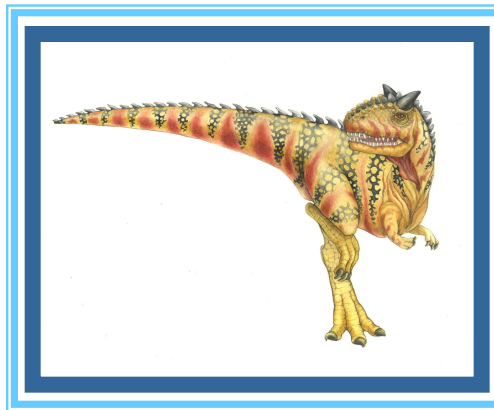


Chapter 2: Operating-System Structures





Chapter 2: Operating-System Structures

- Operating System Services
- User and Operating System-Interface
- System Calls
- System Services
- Linkers and Loaders
- Why Applications are Operating System Specific
- Operating-System Design and Implementation
- Operating System Structure
- Building and Booting an Operating System
- Operating System Debugging

Objectives

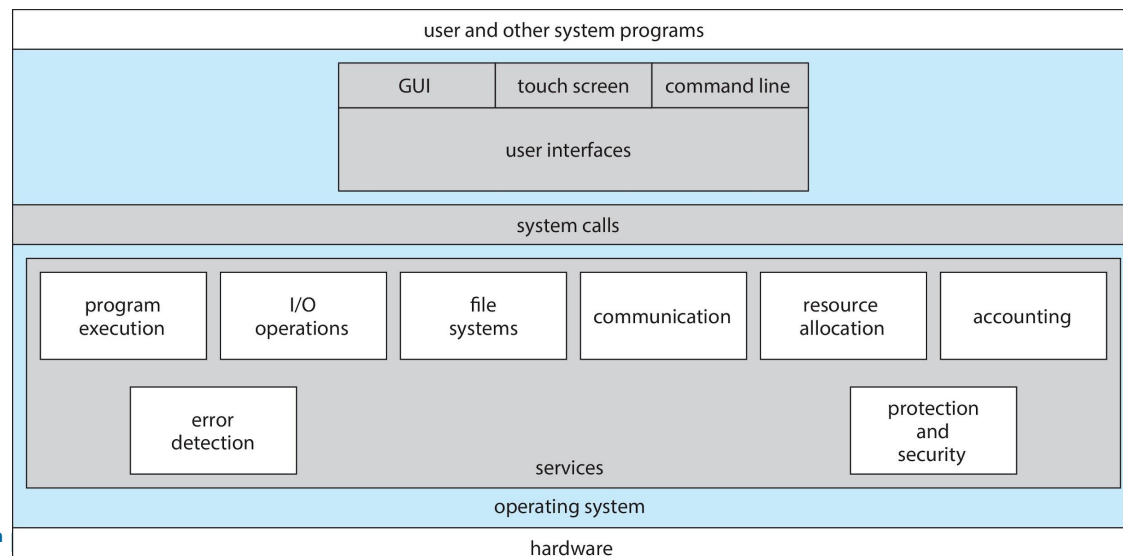
- Identify services provided by an operating system
- Illustrate how system calls are used to provide operating system services
- Compare and contrast monolithic, layered, microkernel, modular, and hybrid strategies for designing operating systems
- Illustrate the process for booting an operating system
- Apply tools for monitoring operating system performance
- Design and implement kernel modules for interacting with a Linux kernel





Operating System Services

- Operating systems provide an environment for execution of **programs** and **services** to programs and users
- One set of operating-system services provides functions that are helpful to the user:
 - User interface** - Almost all operating systems have a user interface (**UI**).
 - Varies between **Command-Line (CLI)**, **Graphics User Interface (GUI)**, **touch-screen**, **Batch**
 - Program execution** - The system must be able to load a program into memory and to run that program, end execution, either normally or abnormally (indicating error)
 - load a program into memory
 - run that program
 - end execution
 - either normally or abnormally (indicating error)
 - I/O operations** - A running program may require I/O, which may involve a file or an I/O device





User Operating System Interface - CLI

CLI (Command-Line Interface) is a way to interact with a computer by **typing commands** instead of clicking

CLI is also called **command interpreter**

- Sometimes implemented (part) in kernel, sometimes by systems program (like a shell)
- Sometimes multiple flavors implemented – **shells**
 - 4 There can be different versions of CLI called **shells**
 - 4 **Examples:** Bash (Linux) PowerShell (Windows)
 - Each shell has slightly different commands/features
- Primarily fetches a command from user and executes it
 - 4 **Example:** **mkdir test** : CLI tells OS to create a folder
- Sometimes commands built-in (Already inside the shell), sometimes just names of programs
 - 4 If the latter, adding new features doesn't require shell modification





Bourne Shell Command Interpreter

What is Bourne Shell?

- The **Bourne Shell** is one of the **first command interpreters (shells)** created for Unix
- It allows users to **type commands** and the OS executes them

```
1. root@r6181-d5-us01:~ (ssh)
X root@r6181-d5-u...  #1 X ssh #2 X root@r6181-d5-us01... #3
Last login: Thu Jul 14 08:47:01 on ttys002
iMacPro:~ pbg$ ssh root@r6181-d5-us01
root@r6181-d5-us01's password:
Last login: Thu Jul 14 06:01:11 2016 from 172.16.16.162
[root@r6181-d5-us01 ~]# uptime
 06:57:48 up 16 days, 10:52,  3 users,  load average: 129.52, 80.33, 56.55
[root@r6181-d5-us01 ~]# df -kh
Filesystem      Size  Used Avail Use% Mounted on
/dev/mapper/vg_ks-lv_root
                  50G   19G   28G   41% /
tmpfs            127G  520K  127G    1% /dev/shm
/dev/sda1        477M   71M  381M   16% /boot
/dev/dssd0000    1.0T  480G  545G   47% /dssd_xfs
tcp://192.168.150.1:3334/orangeufs
                  12T  5.7T  6.4T   47% /mnt/orangeufs
/dev/gpfs-test   23T  1.1T  22T    5% /mnt/gpfs
[root@r6181-d5-us01 ~]#
[root@r6181-d5-us01 ~]# ps aux | sort -nrk 3,3 | head -n 5
root      97653 11.2  6.6 42665344 17520636 ?    S<    Ll  Jul13 166:23 /usr/lpp/mmfs/bin/mmfstd
root      69849  6.6  0.0      0      0 ?        S      Jul12 181:54 [vpthread-1-1]
root      69850  6.4  0.0      0      0 ?        S      Jul12 177:42 [vpthread-1-2]
root       3829  3.0  0.0      0      0 ?        S      Jun27 730:04 [rp_thread 7:0]
root       3826  3.0  0.0      0      0 ?        S      Jun27 728:08 [rp_thread 6:0]
[root@r6181-d5-us01 ~]# ls -l /usr/lpp/mmfs/bin/mmfstd
-r-x----- 1 root root 20667161 Jun  3  2015 /usr/lpp/mmfs/bin/mmfstd
[root@r6181-d5-us01 ~]#
```





User Operating System Interface - GUI

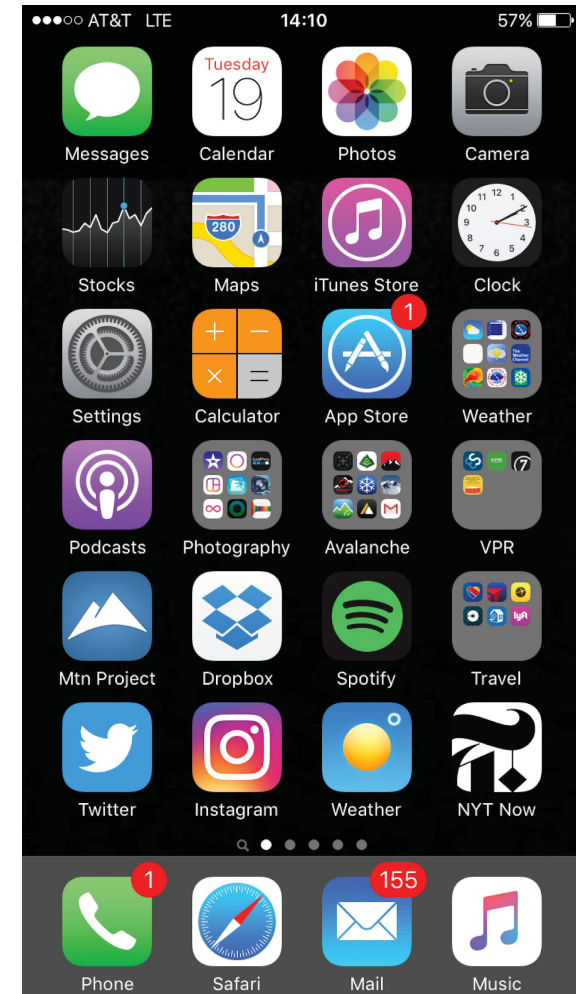
- User-friendly **desktop** metaphor(Mac) interface
 - Usually mouse, keyboard, and monitor
 - **Icons** represent files, programs, actions, etc
 - Various mouse buttons over objects in the interface cause various actions (provide information, options, execute function, open directory (known as a **folder**))
 - Invented at Xerox PARC
 - 4 GUI concept was first developed at **Xerox PARC**
 - 4 Later used in Windows and Mac
- Many systems now include both CLI and GUI interfaces
 - Microsoft Windows is GUI with CLI “command” shell
 - Apple Mac OS X is “Aqua” GUI interface with UNIX kernel underneath and shells available
 - Unix and Linux have CLI with optional GUI interfaces (CDE, KDE, GNOME)





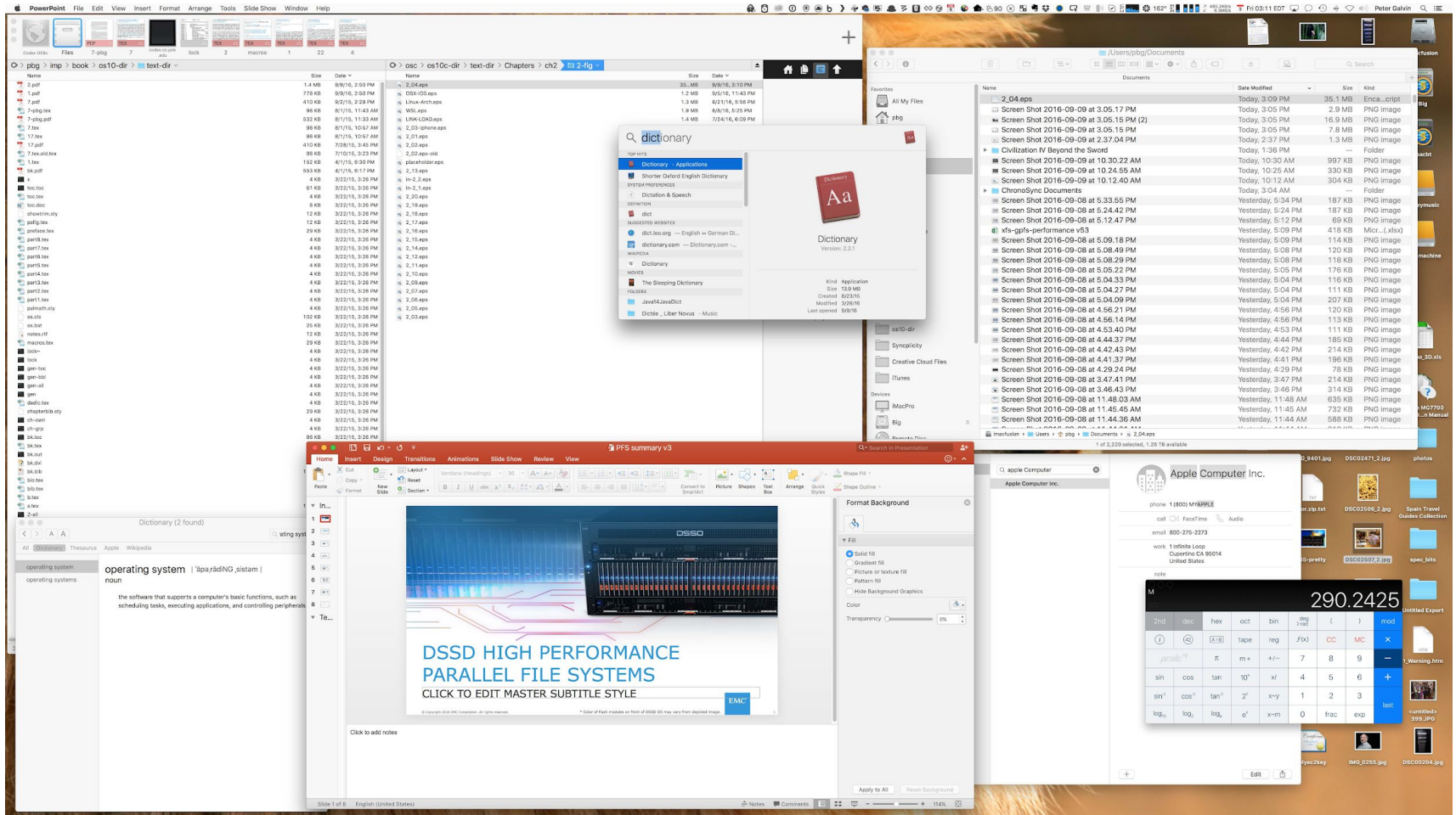
Touchscreen Interfaces

- Touchscreen devices require new interfaces
 - **Old Version Mouse not possible or not desired**
 - Actions and selection based on **gestures**
 - Virtual keyboard for text entry
- Voice commands





The Mac OS X GUI





System Calls

- Programming interface to the services provided by the OS
 - A system call is a way for a program to request services from the Operating System (OS)
- System calls act like a **bridge** between
 - Application (your program)
 - Operating System
- Typically written in a high-level language (C or C++, Python ,Java ,C#, PHP)
- Mostly accessed by programs via a high-level **Application Programming Interface (API)** rather than direct **system call use**
- Three most common APIs are Win32 API for Windows, POSIX API for POSIX-based systems (including virtually all versions of UNIX, Linux, and Mac OS X), and Java API for the Java virtual machine (JVM)

Note that the system-call names used throughout this text are generic





Example of Standard API

EXAMPLE OF STANDARD API

As an example of a standard API, consider the `read()` function that is available in UNIX and Linux systems. The API for this function is obtained from the `man` page by invoking the command

```
man read
```

on the command line. A description of this API appears below:

```
#include <unistd.h>

ssize_t read(int fd, void *buf, size_t count)
```

<code>ssize_t</code>	<code>read</code>	<code>(int fd, void *buf, size_t count)</code>
return value	function name	parameters

A program that uses the `read()` function must include the `unistd.h` header file, as this file defines the `ssize_t` and `size_t` data types (among other things). The parameters passed to `read()` are as follows:

- `int fd`—the file descriptor to be read
- `void *buf`—a buffer into which the data will be read
- `size_t count`—the maximum number of bytes to be read into the buffer

On a successful read, the number of bytes read is returned. A return value of 0 indicates end of file. If an error occurs, `read()` returns `-1`.

```
# Open file
file = open("example.txt", "r")

# Read data
data = file.read(100)  # read up to 100 characters

# Print data
print(data)

# Close file
file.close()
```

Your Python Code



Python API (`open`, `read`, `close`)



System Calls (OS level)



Operating System



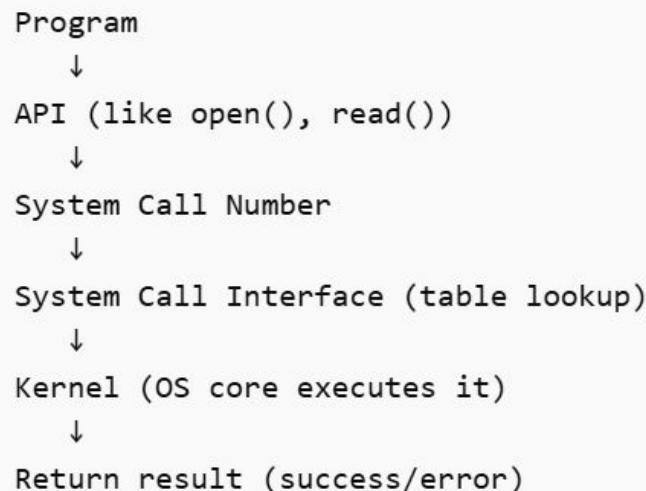
Hardware (disk)





System Call Implementation

- Typically, a number associated with each system call
 - **Each system call has a number**
 - 4 Every system call (like read, write, open) has a **unique ID (number)**
 - 4 Example : **read** → **3** , **write** → **4** , **open** → **5**
 - **System-call interface** maintains a table indexed according to these numbers
- The system call interface invokes the intended system call in OS kernel and returns status of the system call and any return values
- The caller need know nothing about how the system call is implemented
 - Just needs to obey API and understand what OS will do as a result call
 - Most details of OS interface hidden from programmer by API
 - 4 Managed by run-time support library (set of functions built into libraries included with compiler)





System Call Parameter Passing

- Often, more information is required than simply identity of desired system call
 - Exact type and amount of information vary according to OS and call
- Three general methods used to pass parameters to the OS
 - **Simplest: pass the parameters in registers**
 - Parameters are stored in **CPU registers**
 - 4 In some cases, may be more parameters than registers
 - Parameters stored in a block, or table, in memory, and address of block passed as a parameter in a register
 - 4 This approach taken by Linux and Solaris
 - Parameters placed, or **pushed**, onto the **stack** by the program and **popped** off the stack by the operating system
 - Block and stack methods do not limit the number or length of parameters being passed





Types of System Calls

- **1.Process control**

- create process, terminate process
- end, abort
- load, execute
- get process attributes, set process attributes
- wait for time
- wait event, signal event
- allocate and free memory
- Dump memory if error
- **Debugger** for determining **bugs, single step** execution
- **Locks** for managing access to shared data between processes

- **2.File management**

- create file, delete file
- open, close file
- read, write, reposition
- get and set file attributes





Types of System Calls (cont.)

- **3.Device management**
 - request device, release device
 - read, write, reposition
 - get device attributes, set device attributes
 - logically attach or detach devices
- **4.Information maintenance**
 - get time or date, set time or date
 - get system data, set system data
 - get and set process, file, or device attributes
- **5.Communications**
 - create, delete communication connection
 - send, receive messages if **message passing model** to **host name** or **process name**
 - 4 From **client** to **server**
 - **Shared-memory model** create and gain access to memory regions
 - transfer status information
 - attach and detach remote devices
- **6.Protection**
 - Control access to resources
 - Get and set permissions
 - Allow and deny user access





Examples of Windows and Unix System Calls

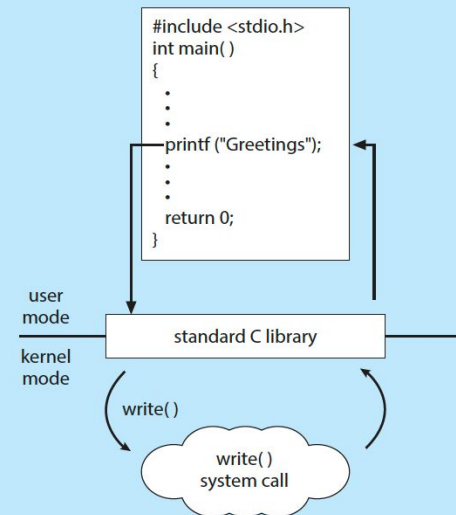
EXAMPLES OF WINDOWS AND UNIX SYSTEM CALLS

The following illustrates various equivalent system calls for Windows and UNIX operating systems.

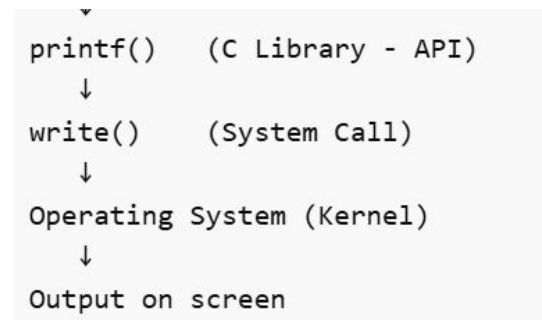
	Windows	Unix
Process control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File management	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device management	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communications	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shm_open() mmap()
Protection	SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()

THE STANDARD C LIBRARY

The standard C library provides a portion of the system-call interface for many versions of UNIX and Linux. As an example, let's assume a C program invokes the `printf()` statement. The C library intercepts this call and invokes the necessary system call (or calls) in the operating system—in this instance, the `write()` system call. The C library takes the value returned by `write()` and passes it back to the user program:



C program invoking `printf()` library call, which calls `write()` system call





System Services

- System programs provide a convenient environment for program development and execution. They can be divided into:
 1. File manipulation
 2. Status information sometimes stored in a file
 3. Programming language support
 4. Program loading and execution
 5. Communications
 6. Background services
 7. Application programs
- Most users' view of the operation system is defined by system programs, not the actual system calls
- **File management** - Create, delete, copy, rename, print, dump, list, and generally manipulate files and directories
- **Status information**
 - Some ask the system for info - date, time, amount of available memory, disk space, number of users
 - Others provide detailed performance, logging, and debugging information
 - Typically, these programs format and print the output to the terminal or other output devices
 - Some systems implement a **registry** - used to store and retrieve configuration information



System Services (Cont.)

- **File modification**

- Text editors to create and modify files
- Special commands to search contents of files or perform transformations of the text

- **Programming-language support** - Compilers, assemblers, debuggers and interpreters sometimes provided

- **Program loading and execution**- Absolute loaders, relocatable loaders, linkage editors, and overlay-loaders, debugging systems for higher-level and machine language

- **Communications** - Provide the mechanism for creating virtual connections among processes, users, and computer systems

- Allow users to send messages to one another's screens, browse web pages, send electronic-mail messages, log in remotely, transfer files from one machine to another

- **Background Services**

- Launch at boot time
 - 4 Some for system startup, then terminate
 - 4 Some from system boot to shutdown
- Provide facilities like disk checking, process scheduling, error logging, printing
- Run in user context not kernel context
- Known as **services**, **subsystems**, **daemons**

- **Application programs**

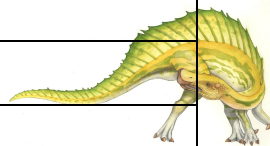
- Don't pertain to system
- Run by users
- Not typically considered part of OS
- Launched by command line, mouse click, finger poke



Linkers and Loaders

- **Source code** compiled into **object files** designed to be loaded into any physical memory location – **relocatable object file** (Not ready to run yet)
- **Linker** combines multiple object files into single binary **executable** file
 - Also brings in libraries
- Program resides on secondary storage as binary executable
 - The executable is saved in Hard disk / SSD (secondary storage)
- Must be brought into memory by **loader** to be executed
 - **Relocation** assigns final addresses to program parts and adjusts code and data in program to match those addresses
- Modern general-purpose systems don't link libraries into executables
 - Rather, **dynamically linked libraries** (in Windows, **DLLs**) are loaded as needed, shared by all that use the same version of that same library (loaded once)
- Object, executable files have standard formats, so operating system knows how to load and start them

Term	Meaning
Object File	Compiled code (not yet runnable)
Linker	Combines files into executable
Executable	Final program
Loader	Loads program into memory
Relocation	Adjust memory addresses
Dynamic Linking	Load libraries when needed





Why Applications are Operating System Specific

- Apps compiled on one system usually not executable on other operating systems
- Each operating system provides its own unique system calls
 - Own file formats, system calls, OS architecture etc
- Apps can be multi-operating system
 - Written in interpreted language like Python, Ruby, and interpreter available on multiple operating systems
 - App written in language that includes a VM containing the running app (like Java)
 - Use standard language (like C), compile separately on each operating system to run on each
- **Application Binary Interface (ABI)** is architecture equivalent of API, defines how different components of binary code can interface for a given operating system on a given architecture, CPU, etc
- **Easy Analogy**
 - OS** = different languages (English, Arabic)
 - Program** = message :
 - Message** must match the language



Operating System Design and Implementation

User goals and System goals

- User goals – operating system should be convenient to use, easy to learn, reliable, safe, and fast
- System goals – operating system should be easy to design, implement, and maintain, as well as flexible, reliable, error-free, and efficient

System Goals Example:

- OS developers aim to design the system **efficiently**, so it runs **fast** while consuming minimal resources.
- Engineers focus on making the OS **modular and maintainable**, so updates and bug fixes can be applied easily.
- Security architects design **error-free** mechanisms to prevent hacking and unauthorized access.





Operating System Design and Implementation (Cont.)

- Important principle to separate

Policy: *What* will be done? Policy decides **WHAT** should happen

- Examples:

- Which process runs first?
- Who gets more CPU time?
- Which user can access a file?
- ✓ It is about **rules / decisions**

Mechanism: *How* to do it?

- Mechanisms determine how to do something, policies decide what will be done

- Examples:

- How CPU switches between processes
- How memory is allocated
- How files are accessed
- ✓ It is about **implementation / method**

- The separation of policy from mechanism is a very important principle, it allows maximum **flexibility** if policy decisions are to be changed later (example – timer)
- Specifying and designing an OS is highly creative task of **software engineering**





Operating System Structure

- General-purpose OS is very large program
- Various ways to structure ones
 - Simple structure – MS-DOS
 - More complex -- UNIX
 - Layered – an abstraction
 - Microkernel -Mach

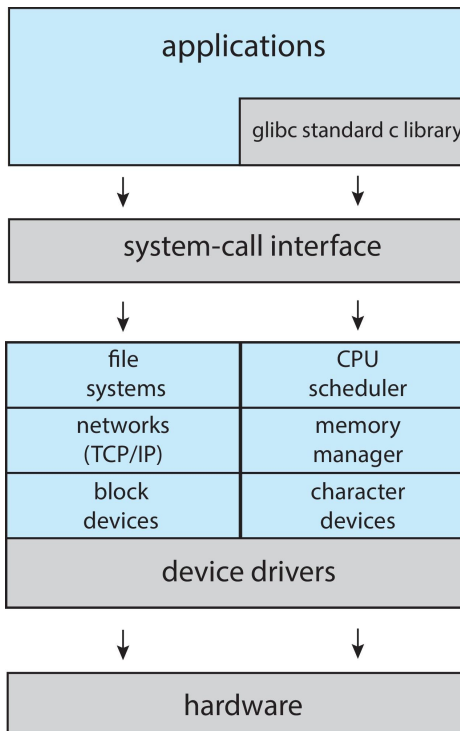




Monolithic Structure – Original UNIX

UNIX – limited by hardware functionality, the original UNIX operating system had limited structuring. The UNIX OS consists of two separable parts

- Systems programs :These are tools users interact with
- The kernel
 - 4 Consists of everything below the system-call interface and above the physical hardware
 - 4 Provides the file system, CPU scheduling, memory management, and other operating-system functions; a large number of functions for one level





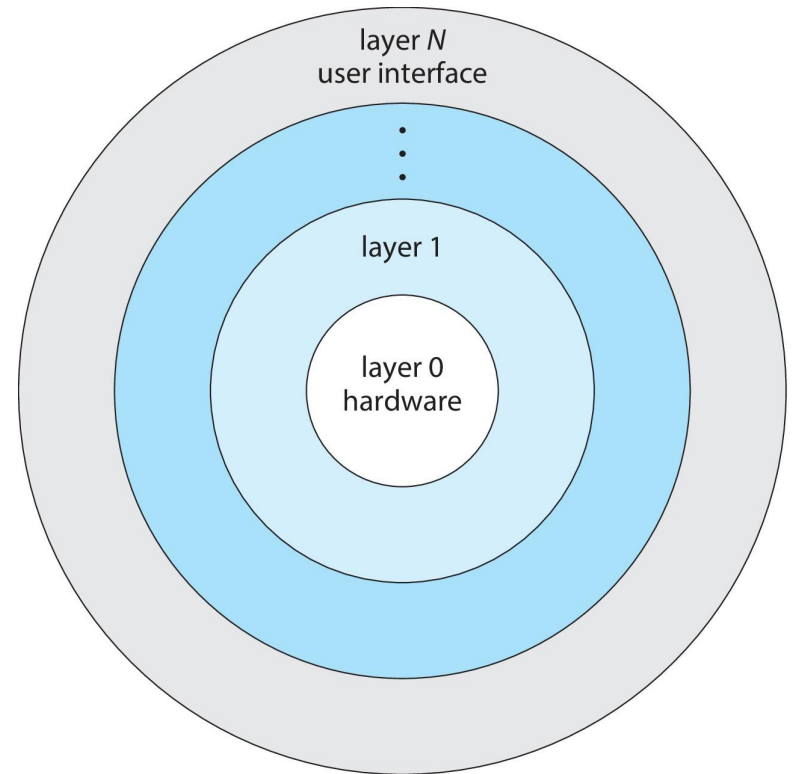
Layered Approach

- The operating system is divided into a number of layers (levels), each built on top of lower layers. The bottom layer (layer 0), is the hardware; the highest (layer N) is the user interface.
- With modularity, layers are selected such that each uses functions (operations) and services of only lower-level layers

Real-Life Analogy

Think of a building:

- Ground floor = hardware
- **First floor (Layer 1)** = workers who directly handle machines
- Upper floors = managers, users





Modules

- Many modern operating systems implement **loadable kernel modules (LKMs)**
- Modern OS (like Linux) use **modules**
- Modules = **small parts of the kernel that can be added or removed when needed**
 - Uses object-oriented approach
 - Each core component is separate
 - Each talks to the others over known interfaces
 - Each is loadable as needed within the kernel
- Overall, similar to layers but with more flexible
 - Linux, Solaris, etc





Building and Booting an Operating System

- Operating systems generally designed to run on a class of systems with variety of peripherals
- Commonly, operating system already installed on purchased computer
 - But can build and install some other operating systems
 - If generating an operating system from scratch
 - 4 Write the operating system source code
 - 4 Configure the operating system for the system on which it will run
 - 4 Compile the operating system
 - 4 Install the operating system
 - 4 Boot the computer and its new operating system





System Boot

- When power initialized on system, execution starts at a fixed memory location
- Operating system must be made available to hardware so hardware can start it
 - Small piece of code – **bootstrap loader**, **BIOS**, stored in **ROM** or **EEPROM** locates the kernel, loads it into memory, and starts it
 - Sometimes two-step process where **boot block** at fixed location loaded by ROM code, which loads bootstrap loader from disk
 - Modern systems replace BIOS with **Unified Extensible Firmware Interface (UEFI)**
- Common bootstrap loader, **GRUB**, allows selection of kernel from multiple disks, versions, kernel options
- Kernel loads and system is then **running**
- Boot loaders frequently allow various boot states, such as single user mode

```
Power ON
↓
BIOS / UEFI
↓
Bootstrap Loader
↓
Boot Loader (GRUB)
↓
Kernel Loaded
↓
Operating System Running
```

Term	Meaning
BIOS	Old firmware
UEFI	Modern firmware
Bootstrap Loader	Loads kernel
GRUB	Boot manager
Kernel	Core of OS





Operating-System Debugging

- **Debugging** is finding and fixing errors, or **bugs**
- Also **performance tuning**
- OS generate **log files** containing error information
- Failure of an application can generate **core dump** file capturing memory of the process
- Operating system failure can generate **crash dump** file containing kernel memory
- Beyond crashes, performance tuning can optimize system performance
 - Sometimes using ***trace listings*** of activities, recorded for analysis
 - **Profiling** is periodic sampling of instruction pointer to look for statistical trends

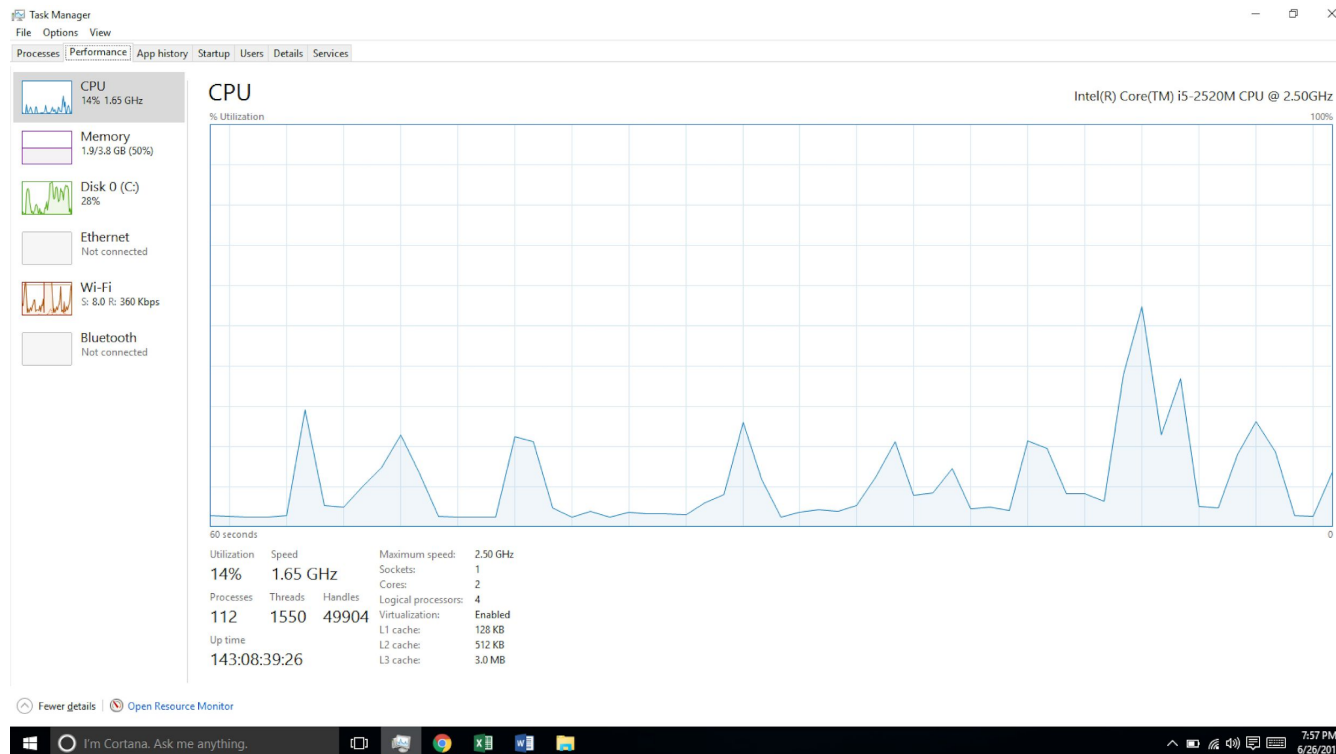
Kernighan's Law: "Debugging is twice as hard as writing the code in the first place. Therefore, if you write the code as cleverly as possible, you are, by definition, not smart enough to debug it."





Performance Tuning

- Improve performance by removing bottlenecks
- OS must provide means of computing and displaying measures of system behavior
- For example, “top” program or Windows Task Manager



End of Chapter 2

